L'Architecture Client-Serveur et le Protocole HTTP

Thème principal : Réseaux et Protocoles Web

Synthèse exécutive : Ce cours pose les fondements indispensables du fonctionnement du Web en détaillant l'architecture client-serveur et le protocole HTTP (HyperText Transfer Protocol). L'étudiant y découvre comment un navigateur (client) formule des requêtes standardisées pour obtenir des ressources auprès d'un serveur web, comment ce dernier y répond de manière asynchrone et sans état (stateless), et comment décoder les codes de statut HTTP (2xx, 3xx, 4xx, 5xx) pour diagnostiquer et déboguer efficacement des applications web.

L'Architecture Client-Serveur et le Principe de Communication

L'architecture client-serveur est le modèle de communication fondamental du Web. Contrairement à un réseau pair-à-pair (P2P) où chaque machine est à la fois cliente et serveuse, ce modèle établit une distinction stricte des rôles. Le client (le navigateur web de l'utilisateur, comme Firefox, Chrome ou Safari) initie toujours la communication en envoyant une requête HTTP à travers le réseau. Le serveur web (comme Apache, Nginx ou IIS), quant à lui, reste dans un état d'écoute passive. Dès qu'il reçoit une requête, il la traite, accède éventuellement à des ressources locales ou à une base de données, puis renvoie une réponse HTTP contenant la ressource demandée (HTML, CSS, image, JSON) ou un message d'erreur.

- Client (Navigateur) : Logiciel actif qui prend l'initiative d'envoyer une requête pour obtenir une ressource.
- Serveur Web : Processus logiciel passif qui écoute sur un port spécifique (ex: 80 pour HTTP, 443 pour HTTPS) et traite les requêtes entrantes.
- Communication Asynchrone : Le client initie l'échange, le serveur traite la demande puis renvoie la réponse, mettant fin à la transaction.

Cas concret enseignant : *Le professeur explique que lorsqu'un étudiant se connecte sur le site de l'IUT de Bayonne, son navigateur n'envoie pas une seule requête unique, mais des dizaines de requêtes HTTP simultanées pour récupérer le fichier HTML de base, puis chaque image, feuille de style CSS et script JavaScript associé afin de reconstituer la page complète.*

Le Protocole HTTP et sa Nature Stateless (Sans État)

Une caractéristique essentielle du protocole HTTP est qu'il est 'stateless' (sans état). Cela signifie que le serveur ne conserve aucune mémoire des requêtes précédentes d'un même client. Chaque transaction requête-réponse est isolée et autonome. Si cette absence d'état simplifie grandement l'architecture et améliore la scalabilité des serveurs (qui n'ont pas à maintenir des millions de connexions ouvertes), elle pose un problème pour les applications modernes qui nécessitent un suivi de l'utilisateur (comme le maintien d'une session de connexion ou la gestion d'un panier d'achat). Pour pallier cette limite, des mécanismes complémentaires comme les cookies et les sessions côté serveur ont été introduits.

- Protocole Stateless (Sans état) : Caractéristique fondamentale de HTTP où chaque échange requête-réponse est indépendant, sans mémoire des interactions passées.
- Mécanismes de Session : Utilisation de cookies (stockés sur le client) et de sessions (gérées côté serveur) pour simuler un état persistant.

Cas concret enseignant : *Le professeur compare HTTP à un guichetier amnésique : chaque fois que vous vous présentez devant lui, même si c'était il y a deux secondes, il vous traite comme un parfait inconnu. Pour qu'il se souvienne de vous, vous devez lui présenter un badge (le cookie de session) à chaque phrase.*

Structure des Messages HTTP : Requêtes, Réponses et Méthodes

Une requête HTTP est un message textuel structuré comportant une première ligne descriptive, des en-têtes (headers), une ligne vide obligatoire, et un corps (body) optionnel. La réponse du serveur adopte une structure

similaire, mais commence par une 'ligne de statut'. Les deux méthodes les plus courantes sont GET et POST. GET est utilisée pour demander ou récupérer une ressource sans modifier l'état du serveur. POST est utilisée pour soumettre des données destinées à modifier l'état du serveur (création d'un compte, envoi d'un formulaire), les données transitant de manière invisible dans le corps de la requête.

- Ligne de requête : Contient la méthode (GET, POST), l'URI de la ressource et la version du protocole.
- En-têtes (Headers) : Métadonnées sous forme de clés-valeurs décrivant le client, le serveur ou le contenu de la transaction.
- Corps (Body) : Zone optionnelle contenant les données brutes envoyées au serveur (ex: données de formulaire, JSON).
- Méthodes GET vs POST : GET est pour la consultation (paramètres dans l'URL, sûre et idempotente), POST est pour la modification (paramètres dans le corps).

Cas concret enseignant : *Le professeur insiste sur le fait que le protocole HTTP est purement textuel : on peut littéralement lire les requêtes et réponses en clair si elles ne sont pas chiffrées par HTTPS. C'est pourquoi l'utilisation de GET pour des données sensibles comme un mot de passe est une faille de sécurité majeure visible directement dans l'historique du navigateur ou sur l'écran de n'importe qui passant derrière l'utilisateur (shoulder surfing).*

La Classification des Codes de Statut HTTP

Chaque réponse HTTP renvoyée par le serveur commence par une ligne de statut contenant un code numérique à trois chiffres. Ce code permet au client de comprendre instantanément le résultat de sa demande. Il est crucial pour un développeur de faire la distinction entre les erreurs de la famille 4xx (problème imputable au client, ex: URL erronée) et 5xx (problème interne au serveur, ex: script défectueux ou base de données inaccessible).

- Codes 2xx (Succès) : Indiquent que la requête a été reçue, comprise et acceptée avec succès (ex: 200 OK).
- Codes 3xx (Redirection) : Indiquent que des actions supplémentaires sont requises pour compléter la requête (ex: 301 Moved Permanently).
- Codes 4xx (Erreur Client) : Indiquent une anomalie dans la requête formulée par le client (ex: 404 Not Found, 403 Forbidden).
- Codes 5xx (Erreur Serveur) : Indiquent que le serveur a échoué à traiter une requête pourtant valide (ex: 500 Internal Server Error).

Cas concret enseignant : *Le professeur raconte qu'à ses débuts, il avait configuré par erreur une base de données qui plantait à chaque soumission de formulaire, renvoyant systématiquement une erreur 500. Les utilisateurs pensaient que le site était piraté alors que c'était simplement un point-virgule manquant dans son script PHP.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
Réponse du professeur : Le professeur a expliqué que GET envoie les paramètres directement dans l'URL (visibles dans la barre d'adresse et l'historique, taille limitée), tandis que POST les cache dans le corps (body) de la requête. Pour un formulaire de connexion contenant un mot de passe, l'utilisation de GET est une faille de sécurité critique, bien que POST nécessite également HTTPS pour chiffrer les données sur le réseau.

■ Exercice & Démonstration d'Application

Énoncé : Analyser la requête HTTP brute suivante et identifier la méthode utilisée, la ressource demandée, l'hôte cible, et expliquer ce que le serveur va renvoyer si la ressource n'existe pas : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0

Résolution pas à pas :
1. Analyse de la ligne de requête : La première ligne est 'GET /images/logo.png HTTP/1.1'. Le premier mot est le verbe ou la méthode HTTP, ici 'GET', qui indique une demande de récupération de ressource sans modification d'état. La ressource demandée (URI) est '/images/logo.png'. 2. Identification de l'hôte : L'en-tête 'Host:

iutbayonne.univ-pau.fr' spécifie le nom de domaine du serveur auquel la requête est adressée. 3. Analyse de l'agent utilisateur : L'en-tête 'User-Agent: Mozilla/5.0' identifie le navigateur client. 4. Scénario d'absence de ressource : Si le fichier 'logo.png' n'existe pas dans le répertoire du serveur web, celui-ci ne peut pas traiter la demande avec succès et doit renvoyer un code de statut de la classe 4xx.

Résultat : Méthode : GET Ressource : /images/logo.png Hôte cible : iutbayonne.univ-pau.fr Code de retour en cas d'absence : 404 Not Found (Erreur Client)

■ Auto-Évaluation Flash d'Examen

Q : Qu'est-ce qu'un protocole 'stateless' comme HTTP ?

R : C'est un protocole dans lequel chaque échange requête-réponse est totalement indépendant et le serveur ne conserve aucune mémoire des interactions passées.

Q : Quelle est la différence majeure entre les codes d'erreur de la série 4xx et ceux de la série 5xx ?

R : Les erreurs 4xx indiquent un problème imputable au client (ex: URL fausse, ressource absente), tandis que les erreurs 5xx indiquent une défaillance interne du serveur (ex: crash de script ou de base de données).

Q : Pourquoi n'utilise-t-on jamais la méthode GET pour transmettre un mot de passe ?

R : Parce que GET intègre les paramètres directement dans l'URL, ce qui les rend visibles dans la barre d'adresse, l'historique du navigateur et les logs du serveur, constituant une faille de sécurité majeure.